

LLMがカルテ情報に基づいて、治験基準に関する23問に回答とその根拠を示す研究をする。治験基準に関する23問について、専門家によるアノテーションがある。どのようなパイプラインで一番精度の高い回答ができるのをタスクとする。ここの「回答」は、治験基準に対する回答。例えば、「この診療記録には、患者がこれまでに双極性障害と診断されたことがあると記載されていますか？」という問題に対して、「はい」（記載している）、「いいえ」（診断されたことがない）と明記している、あるいは精神疾患なしなど、似たような意味を持つ記載がある）、「記載なし」

表示を増やす

1m 20s考えました

12m 17s考えました

結論として、この研究は十分実行可能です。ただし、現在の6案をそのまま比較するよりも、タスクと正解ラベルを整理し、共通の「原文根拠管理」と「数値計算モジュール」を全パイプラインに持たせる必要があります。

現時点で最も高精度になる可能性が高いのは、修正版のパイプライン5です。

高再現率のセクション検索＋異種LLMによる独立回答＋根拠制約付き集約＋全文検証＋決定論的な数値集計

一方、パイプライン6の討論型エージェントは、計算量が大きいわりにパイプライン5を安定して上回るとは限りません。実運用では「パイプライン4を通常経路、低信頼例だけパイプライン5、さらに解決しない例を人間へ」が最も合理的です。

1. 添付CSVから分かった重要事項

添付の `annotated_apixaban_combined.csv` を確認した結果は次のとおりです。

項目	確認結果
----	------

研究パイプライン比較・Work

チャットを PDF に変換 | 共有する | ...

質問数	23
Q&A総数	2,300
記載系	15問・1,500回答
数値系	8問・800回答
1記録当たり質問数	全例23
記録長	976～4,310語、中央値2,024語
数値回答欠損	267
not_specified=1	265
欠損とフラグの不整合	2件（LVEF、bilirubin）

進捗

- 設計仕様・実装計画を確定する
- テスト先行で共通データ・根拠・数値処理を実装する
- LLMアダプター、ルータ、P1～P6を実装する
- CLI・評価・設定例・ドキュメントを実装する
- コードレビューと全テスト・実データdry-runを行う
- PHIを除外して配布用アーカイブを作成する

出力

- ファイル/サイトを作成

項目	確認結果
原文根拠・section・rationale列	なし

このデータは、PhysioNetが公開する100記録×23問の人手確認済みデータと一致します。
ただし、公開データが持つ正解は、原則として15問がYes/No、8問が数値/NAです。
[PhysioNet公式データカード](#) ▶

特に重要なのは次の3点です。

- 現在のYes/No正解では、「明示的な否定」と「記載なし」が区別されていません。
- 希望されている「不明」ラベル也没有。
- 根拠となる原文スパンや、数値に関する全候補値の正解也没有。

したがって、研究を二段階に分ける必要があります。

- 既存ベンチマーク評価：元のYes/No・数値/NAに対する精度
- 拡張評価：Yes/No/記載なし/不明、根拠スパン、全数値候補を新たに専門家注釈して評価

根拠注釈を追加しない場合、「回答精度」は評価できますが、「提示した根拠が正しい」「すべての数値を抽出できた」という主張はできません。

2. この研究の正確なタスク名称

このデータだけを使用する段階では、研究を「患者-治験マッチング」と呼ぶよりも、次のように位置づけるのが正確です。

Evidence-grounded criterion-level clinical trial question answering from clinical notes
診療記録に基づく、原文根拠付き治験基準別QA

23問は固定されており、複数治験の検索・ランキングや、最終的な治験適格性そのものを評価しているわけではないからです。

3. 回答ラベルの再定義

3.1 記載系15問

以下の4状態にします。

状態	定義
Yes	患者が条件に該当すると明確に記載
No	該当しないことが明示されている、または包括的な否定表現がある
Not documented	関連情報が記録内に存在しない
Indeterminate	関連記載はあるが、具体性不足、矛盾、可能性表現などにより判定不能

例えば、双極性障害について：

- “History of bipolar disorder” → Yes
- “No psychiatric history” → No
- 精神疾患関連記載なし → Not documented
- “History of psychiatric illness”のみ → Indeterminate

「No」と「Not documented」は必ず分けるべきです。LongHealthでも、長い臨床文書における「情報が存在しない」判定はLLMが特に苦手と報告されています。[LongHealth](#) ▶

3.2 数値系8問

数値系に「Yes」を使うと、状態と値が混在するため、次の3状態を推奨します。

状態	定義
Value available	1個以上の有効な数値を抽出可能

サブエージェント



16 件完了

ソース

 ウェブ検索

 ファイル検索

 メモリ

状態	定義
Not documented	検査・項目自体の記載がない
Indeterminate	検査実施の記載はあるが結果不明、単位不明、値が曖昧など

さらに、状態とは別に、以下を出力します。

- 全候補値
- 原単位と正規化単位
- 日時
- 原文引用
- 文字位置
- 採用・除外理由
- 最小値／最大値

4. 共通の出力形式

すべてのパイプラインが同じ構造を出すようにします。

<> JSON

```
{
  "question_id": "PLT",
  "document_status": "value_available",
  "final_value": 85,
  "unit": "10^9/L",
  "candidate_values": [
    {
      "raw_value": "120",
      "normalized_value": 120,
      "quote": "PLT 120 K/uL",
      "start_char": 15420,
      "end_char": 15433,
      "section": "Laboratory Results",
      "time": "recorded date"
    },
    {
      "raw_value": "85",
      "normalized_value": 85,
      "quote": "Platelets decreased to 85",
      "start_char": 18201,
      "end_char": 18226,
      "section": "Hospital Course",
      "time": "recorded date"
    }
  ],
  "inference_summary": "Among the two documented platelet values, 85 is the minimum.",
  "confidence": 0.94
}
```

重要なのは、`quote` と `inference_summary` を別フィールドにすることです。

原文引用については、プログラムで必ず

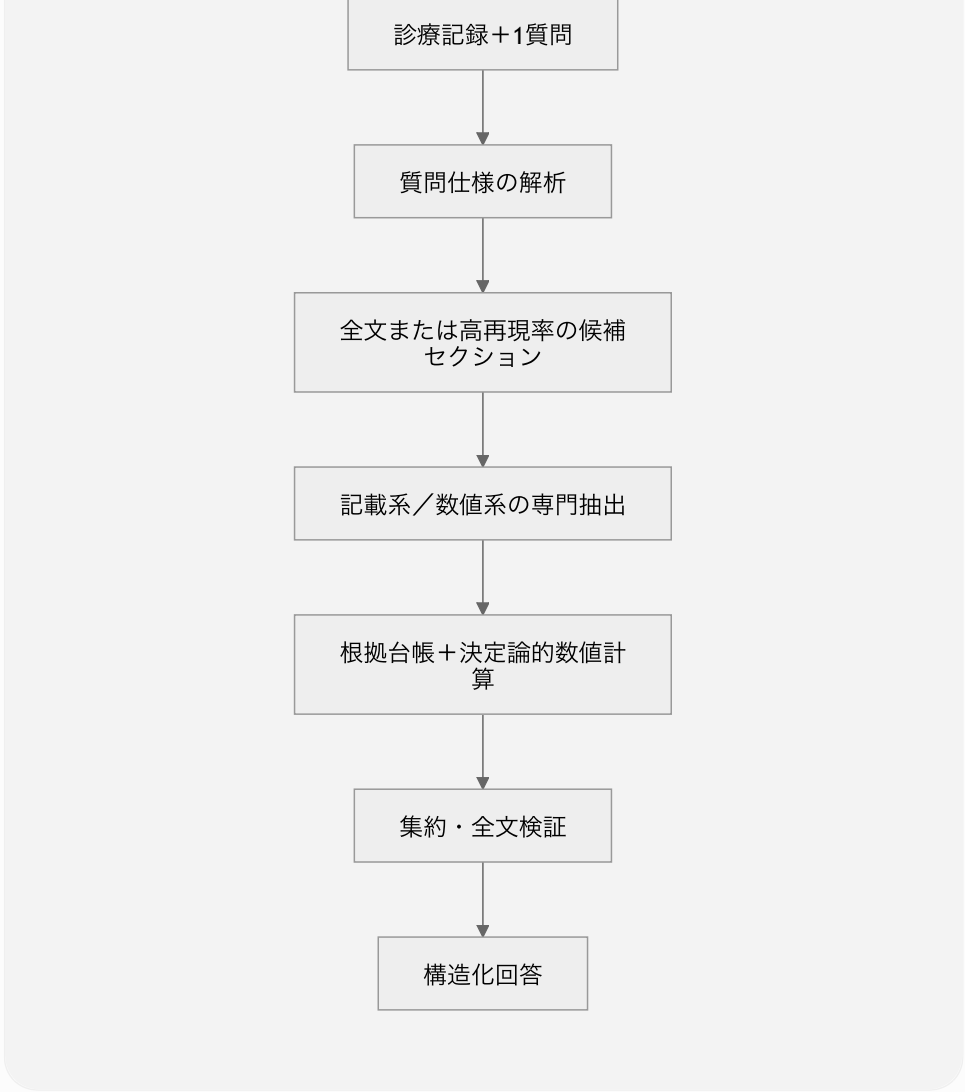
$$D_i[\text{start} : \text{end}] = \text{quote}$$

を検証します。LLMが生成した「もっともらしい引用」は認めません。

5. 推奨する共通パイプライン

Mermaid

</> ▶ ↺



数値系はLLMに最小値・最大値を直接計算させない

数値系では、LLMの役割を「全候補値の抽出と意味確認」に限定します。

$$Z_{ij} = \{(x_k, u_k, t_k, s_k)\}_{k=1}^K$$

ここで、 x_k は値、 u_k は単位、 t_k は時点、 s_k は原文スパンです。

単位正規化後、プログラムで

$$\hat{v}_{ij} = g_j \left(\{\text{normalize}(x_k, u_k) : x_k \in Z_{ij}^{\text{valid}}\} \right), \quad g_j \in \{\min, \max\}$$

を計算します。

Wooらの原著でも、検査値に対して直接Yes/Noを尋ねるより、「数値抽出→ルールによる後処理」の方が高精度でした。また、NA例と原文根拠を学習に含めることが重要でした。Woo et al., npj Digital Medicine 2025

LVEFの「最低値が55%以上なら55と回答」のような特殊ルールも、LLMではなく質問別ルールとして実装します。

6. 6つのパイプラインの整理

6案は、次の3因子で表現できます。

- 入力：全文／ルーティングされたセクション
- 推論主体：単一モデル／異種複数モデル／同一モデル内役割
- 最終検証：なし／全文検証あり

パイプライン	入力	回答主体	最終処理	主な利点	主なリスク
P1	全文	単一LLM	なし	最も単純で強い基準線	単一モデルの見落とし

パイプライン	入力	回答主体	最終処理	主な利点	主なリスク
P2	全文	異種複数LLM	集約LLM	モデルごとの得意分野を利用	関連した誤り、コスト
P3	全文	同一モデル内の複数役割	討論	時間・否定・根拠を別視点で確認	偽の合意、自己強化
P4	関連セクション	単一LLM	全文検証	ノイズ削減、根拠提示しやすい	ルータの見落とし
P5	関連セクション	異種複数LLM	集約＋全文検証	精度上限が最も高い可能性	最も複雑
P6	関連セクション	専門役割エージェント	討論＋全文検証	難例を多面的に確認	高コスト、討論が必ずしも有効でない

P1：全文＋単一LLM

添付データの記録は最大でも4,310語なので、現在の長文対応モデルでは全文入力が十分可能です。したがって、P1は弱いベースラインではなく、かなり強い比較対象になります。

同じ記録に23問を一問ずつ入力し、15問用と8問用でプロンプトを分けます。精度を最優先する主解析では一問ずつ実行し、23問一括処理はコスト評価用の副解析にします。

複数質問を一括処理すると費用を大きく削減できますが、質問数・文書量の増加に伴って回答欠落や精度低下が生じ得ます。[Klang et al., 2024](#) ➤

P2：全文＋異種複数LLM

各モデルは、他モデルの回答を見ずに独立回答します。その後、以下を満たす候補だけを集約します。

- 引用が原文中に実在する
- 引用が回答を支持する
- 時点、否定、患者本人か家族かが適切
- 数値の場合、全候補が列挙されている

同じモデルを複数回呼ぶ場合は「複数LLM」ではなく、self-consistency条件として別に扱います。[Self-Consistency](#) ➤

異種モデルによる討論とconfidence-weighted votingには改善報告がありますが、信頼度は開発データで校正する必要があります。[ReConcile](#) ➤

P3：同一モデル内の複数役割

この23問では、薬剤質問はほとんど存在しないため、「薬剤専門家」を固定的に置くのは適切ではありません。疾患領域ではなく、誤りの種類で役割を分ける方がよいです。

- 疾患・状態の記載抽出
- 否定、可能性、家族歴、患者本人の区別
- 時間条件の判定
- 数値・単位・検査名の抽出
- 根拠完全性の監査
- 最終判定

「確信するまで討論」は避け、最大1～2ラウンドで終了します。解決しなければIndeterminateにします。合意したこと自体は正しさの根拠ではありません。

Multi-Agent Debateは、self-consistencyや単純なアンサンブルを安定して上回らず、ハイパーパラメータに敏感という比較研究があります。[Smit et al., ICML 2024](#) [↗](#)

P4：セクションルーティング＋単一LLM＋全文検証

固定23問なので、質問ラベルを毎回LLMに生成させるより、専門家が一度だけ質問仕様表を作る方が再現性と精度が高い可能性があります。

各質問について事前に以下を定義します。

- 対象概念と同義語
- 推奨セクション
- 時間条件
- 否定の扱い
- 許容される推論
- 単位・min/maxルール

セクションは単一ラベルに限定せず、複数選択にします。例えば双極性障害は、既往歴、問題リスト、入院経過、退院診断のどこにでも記載され得ます。

PRISMも長い実EHRをチャンク化して関連情報を検索し、時間順に並べて criterion-level QAを行っています。[PRISM](#) [↗](#)

ただし、P4～P6では検索漏れが最大の危険です。最終全文LLMは「答えを一から作り直す」のではなく、「現在の回答を覆す記載や、抽出漏れた数値がないか」を確認する監査役にします。

P5：ルーティング＋異種複数LLM＋全文検証

精度最優先なら第一候補です。ただし、集約LLMに自由に回答を再生成させるのではなく、次の順序にします。

1. 各モデルが独立して回答・根拠を出す
2. 原文引用の实在性を機械的に確認
3. 検証済み回答のみを集約
4. 全文から反証・見落としを検索
5. 新しい原文根拠がある場合だけ回答を変更

TrialMatchAIも、hybrid retrieval、reranking、criterion-level reasoningを分離し、Met/Not Met/Unclear/Irrelevantを構造化出力しています。[TrialMatchAI](#) [↗](#)

P6：ルーティング＋専門エージェント＋討論＋全文検証

研究条件としては有用ですが、標準システムとして最初から採用する必要はありません。

P5で以下の場合だけP6を起動する選択的設計がよいです。

- モデル間で回答が一致しない
- YesとNoの根拠が併存
- 時間条件を判定できない
- 数値候補の単位・対象検査が曖昧
- verifierが根拠不足を指摘

討論後に新しい原文根拠が提示されなければ、回答変更を認めないルールにします。

7. 現在の6案には交絡がある

現在の設計では、

- P1～P3：全文・検証なし
- P4～P6：ルーティング・全文検証あり

となっています。

そのためP4～P6が高精度でも、「セクション分割が効いたのか」「全文検証が効いたのか」が分かりません。

科学的に最も明確なのは、次の完全要因実験です。

少なくとも追加すべき条件は以下です。

- 全文＋単一モデル＋全文検証
- ルーティング＋単一モデル＋検証なし
- 全文＋複数モデル＋全文検証
- ルーティング＋複数モデル＋検証なし

さらに、根拠注釈後は「正解根拠セクションを与えたoracle retrieval」を置くと、検索器の性能上限を確認できます。

8. 数理的問題設定

患者記録を D_i 、質問を q_j 、 $i = 1, \dots, N$ 、 $j = 1, \dots, 23$ とします。

正解出力を

$$O_{ij} = (y_{ij}, v_{ij}, E_{ij}, Z_{ij})$$

と定義します。

- y_{ij} ：回答状態
- v_{ij} ：数値問の最終値
- E_{ij} ：回答を支持する原文スパン集合
- Z_{ij} ：数値問に関する全候補値集合

パイプライン π は、

$$C_{ij}^\pi = R_\pi(D_i, q_j)$$

で全文または関連部分を選択し、

$$\tilde{O}_{ij}^{(m)} = G_m(C_{ij}^\pi, q_j)$$

で各モデル m が候補回答を生成します。

その後、

$$\bar{O}_{ij} = A_\pi \left(\{\tilde{O}_{ij}^{(m)}\}_{m=1}^M \right)$$

で集約し、

$$\hat{O}_{ij}^\pi = V_\pi(D_i, q_j, \bar{O}_{ij})$$

で全文検証します。

Joint correctness

回答だけでなく根拠も含めた主要指標として、

$$J_{ij}^\pi = \mathbf{1}(\hat{y}_{ij} = y_{ij}) \cdot \mathbf{1}(\hat{v}_{ij} = v_{ij} \text{ if numeric}) \cdot \mathbf{1}(\hat{E}_{ij} \text{ is valid and sufficient})$$

を定義できます。

23問を均等に評価するmacro scoreは、

$$S_\pi = \frac{1}{23} \sum_{j=1}^{23} \frac{1}{N} \sum_{i=1}^N J_{ij}^\pi$$

です。

ただし、現在のCSVには E_{ij} と Z_{ij} がいないため、現時点では回答部分しか評価できません。

9. 評価指標

回答精度

対象	主要指標
元のYes/No	criterion別balanced accuracy、macro-F1、感度、特異度
拡張4状態	macro-F1、各状態のF1、混同行列
数値	exact match、許容誤差付きaccuracy、MAE
回答不能	Not documented/IndeterminateのF1
全体	23問macro score、joint correctness

単純accuracyだけでは、例えば統合失調症がYes 2件、双極性障害がYes 5件しかないため、ほとんどNoと回答するだけで高くなってしまいます。

根拠精度

- exact quote validity
- sentence/span precision、recall、F1
- gold evidence Recall@k
- evidence sufficiency
- unsupported inference率
- fabricated quote率

TrialGPTはcriterion-level回答に加えて関連文位置を評価し、sentence evidence F1 88.6%を報告しています。TrialGPT ↗

数値完全性

- 全候補値のrecall
- 値・検査名の対応精度
- 単位正規化精度
- reference rangeや日付を値と誤認した割合
- min/maxを与えた根拠行の正確性
- 数値抽出誤りと集計誤りの分離

校正と運用指標

- Brier score
- Expected Calibration Error
- risk–coverage curve
- 23問完答率
- JSON妥当率
- 入出力token数
- API費用
- p50/p95 latency
- 再試行率

LLM自身に「何%自信があるか」と尋ねた値をそのまま信用せず、開発データまたはモデル間一致率で校正します。

10. データ分割と統計解析

患者単位で分割する

同じ記録が23行に繰り返されているため、行単位でtrain/test分割すると、同じ本文が両方に入り、重大なデータリークになります。

必ず `note_id` または患者単位で分割します。

最も望ましい設計は以下です。

- 開発・学習：MIMIC-III-Ext-Synthetic-Clinical-Trial-Questions
- 最終テスト：MIMIC-IV-Ext-Apixaban-Trial-Criteria-Questions 100記録

MIMIC-IV側をプロンプト調整に使用する場合は、患者単位のnested cross-validationまたは事前に固定したdevelopment/test分割が必要です。MIMIC-IIIとMIMIC-IVの2データセットは正式名称と役割を混同しないようにします。

パイプライン間比較

同一patient-questionに6~12パイプラインの予測があるため、対応のある比較を行います。

- 患者単位cluster bootstrapによる95%信頼区間
- correctnessのペア比較：McNemar検定
- 6条件以上の全体比較：Cochran's Q
- Holm法による多重比較補正
- 補助解析：患者とcriterionをランダム効果とする交差分類混合モデル

$$\text{logit } P(C_{ijp} = 1) = \beta_0 + \beta_p + \beta_t + u_i + v_j$$

ここで、 p はパイプライン、 t は質問型、 u_i は患者、 v_j はcriterionのランダム効果です。

100記録は全体比較には使用できますが、Yesが2~5件しかないcriterionの感度を安定して評価するには不十分です。criterion別の強い結論を出すなら追加注釈が必要です。

11. 先行研究との位置づけ

- 直接の基準研究であるWooらは、同じ23問で小型fine-tunedモデルを評価し、8B-Allで平均balanced accuracy 0.93、micro-F1 0.94を報告しています。したがって、これを再現対象・強ベースラインに含める必要があります。[Woo et al.](#) [↗](#)
- TrialGPTはRetrieval→criterion matching→trial rankingの3段構成で、条件別ラベル、関連文位置、説明を出しています。[TrialGPT公式ページ](#) [↗](#)
- PRISMは実世界EHRの大量ノートを検索し、時間順の根拠を用いてYes/No/NAを回答しています。[PRISM](#) [↗](#)
- TrialMatchAIはhybrid retrieval、reranking、criterion-level 4状態判定をモジュール化し、各構成要素のablationを行っています。[TrialMatchAI](#) [↗](#)
- 異種モデル協調には改善可能性があります。議論は単純投票やself-consistencyを必ずしも上回りません。[ReConcile](#) [↗](#), [Smit et al.](#) [↗](#)

12. 推奨する実装順序

- 23問の詳細なannotation guidelineを作る
- LVEF・bilirubinの2件の不整合を確認する
- P1とWooらの公開ベースラインを再現する
- 共通の原文引用チェッカーと数値reducerを実装する
- P4でセクション検索と全文検証の効果を測る
- P2とP5で異種モデルアンサンブルを比較する
- 最後にP3・P6の討論型を追加する
- プロンプトとモデル版を固定して、患者単位の最終評価を行う

最終的な推奨

研究用の主比較としては6案を残してよいですが、主張を強くするには次の構成が最適です。

- 最有力パイプライン：修正版P5
- 最良の精度-費用バランス候補：P4
- 必須ベースライン：P1+Wooらのfine-tunedモデル
- 研究的検証条件：P3・P6
- 実運用候補：P4 → 不一致・低信頼例のみP5 → それでも不明なら人間

単に「マルチエージェントが単一LLMより高精度だった」という研究では新規性は限定的です。これを、

四状態の回答可能性判定、原文根拠の完全性、数値候補の全抽出、決定論的min/max計算、全文による見落とし検証を統合したevidence-grounded clinical trial QA benchmark

として設計すれば、方法論的にも臨床的にも、より強い研究になります。

上記推奨される構成を採用します。それぞれのパイプラインに関するコードを作成してください。

承知しました。承認いただいた構成に沿って、そのまま実装を継続しています。現在、共通データ層・根拠検証・決定論の数値処理はレビュー済みで、LLMアダプターとセクションルータのレビュー段階です。この後、P1〜P6本体、CLI・評価、添付CSVでの検証、配布用コード一式まで仕上げます。

Review task3 retry 完了

P1〜P6が同じ条件で比較できるよう、LLM呼び出し履歴には本文ではなく、モデル名・task種別・文字数・遅延・prompt hashのみを残す仕様です。現在、このPHI非露出とルータfallbackの独立レビューを進めています。

レビューで、LLMが誤ったセクションラベルを付けた場合に、正しいヒューリスティック一致を押し出す問題が見つかりました。コードレビュー受領の手順に従い、各指摘を仕様と再現例に照らして確認したうえで、重要なものを回帰テスト付きで修正します。

Task3 llm routing 更新あり

コードレビュー受領スキルを確認しパッチを適用した

修正方針は、静的・ヒューリスティック一致を常に優先し、LLM由来ラベルだけでは全文fallbackを止めないことです。また、JSONが辞書であるだけでは成功扱いせず、taskごとの必須フィールド・型・列挙値をコード側validatorで確認し、不正なら上限付き再試行へ回します。

Rereview task3 fix1 完了

Task3 llm routing 更新あり

臨床試験QAパイプライン報告を確認し計画をレビューした

再レビューで、LLMラベルが同点時の第2順位にまだ使われ、正しい別名一致を押し出す経路が残っていました。LLMラベルを順位計算から完全に除外し、またsection label・proposal ID・nested evidenceも各request固有の許容集合と型で検証する修正を進めています。

Task 3の修正では、promptに書いた制約をコード側にも二重化しています。たとえば、集約器が存在しないproposal IDを選ぶ、section labelerが許可外ラベルを返す、evidence配列の要素が辞書でも必須quoteを欠く、といった出力は成功扱いせず再試行されます。

Rereview task3 fix2 完了

Task3 llm routing 完了

確認済みレポート末尾とコミット、臨床試験QA計画をレビュー

残っていた経路は、`deterministic_labels` が空のとき互換目的で `section.labels` へ戻る処理でした。このfallback自体を削除し、ルーティング順位は不変の決定論ラベルと別名一致だけを見るようにします。LLMラベルは監査用メタデータに限定します。

Rereview task3 fix3 完了

臨床試験QAパイプラインを修正しパッチ適用とコミット確認

共通基盤3層がレビューを通過しました。ここからP1～P6本体を実装します。P1/P2/P3は全文入力、P4/P5/P6はrecall-first routing+全文verifier、P2/P5は独立モデル集約、P3/P6は最大2ラウンドの役割パネルとして、同じ `run_case` 契約へ統一します。

 Task4 pipelines 作業開始

P1～P6の実装では、集約LLMやverifierに自由な新規引用を書かせず、既存proposal/evidence IDの選択だけを許します。verifierが修正案を返す場合も、全文に対する完全一致引用と数値reducerを再通過しない限り採用しません。

 確認したGit変更状況

まず集約器から実装中です。booleanは検証済み根拠を伴う票のみを数え、同票は `indeterminate`、`numeric`は各モデルの検証済み候補を和集合にしてから再度min/maxを計算するため、多数決で極値を失わない設計です。

 Git変更状況を確認した

集約器に続き、役割パネルと全文verifierまで実装が進みました。初期役割呼び出しと反論ラウンドを別taskとして記録し、`max_rounds=2` をコードで強制するため、費用と遅延を後から正確に比較できます。

 Gitの変更状況を確認した

P1～P6のクラスとfactoryまで作成され、現在はtopologyテストを調整中です。1問のLLM出力が壊れても残り22問を継続し、エラー結果には例外クラスだけを残してカルテ本文を含めない挙動も確認しています。

 Review task4 完了

 確認した臨床試験QAパイプライン報告と変更

P1～P6本体の初回実装が完了し、現在は最重要の統合レビュー中です。局所25テストと既存33回帰チェックで、6 topology、数値候補和集合、根拠台帳、最大2反論、1問失敗時の継続を確認しています。

統合レビューでは、単にクラスが存在するだけでなく、各質問あたりの実呼び出し数、P4～P6のverifier一回保証、aggregator/verifierが未知IDを返した際の棄却、数値候補がLLM選択で欠落しないことをコード経路ごとに追っています。

 Task4 pipelines 完了

統合レビューで3点を修正中です。verifier例外時に推論文・数値候補を完全消去すること、集約器やrole panelが例外でもP4～P6はverifierを必ず1回呼び出すこと、P2/P5で同一client objectの重複を拒否することです。併せて `confidence=true` も数値として受理しないようにします。

失敗時の出力は、`answer/unit/evidence/candidate_values/inference` をすべて空にし、固定エラーコードと例外クラスだけを残す形へ統一しています。これにより、カルテ由来引用が結果オブジェクトやJSONLへ残る経路を閉じます。

 Rereview task4 fix1 完了

 臨床試験QAパイプラインをレビューしパッチ適用とコミット確認を実施

P1～P6本体は67回の直接チェックと再レビューを通過しました。最後の実装タスクとして、厳格なJSON設定、PHIを含まないJSONL/manifest、既存goldへの評価、`validate-data/split/run/evaluate` CLI、README、依存なし全テストrunnerを作成します。

 Task5 cli eval 作業開始

CLIの `run` は、予測結果に `note_id/criterion/status/answer/検証済み根拠` を保存しますが、診療記録全文やpromptは保存しません。manifestにはpipeline、seed、prompt

version、model名、call数、時刻、config hashだけを残し、再現性とデータ保護を両立させます。

 Git変更状況を確認した

評価モジュールでは、棄却を誤答扱いに含めたaccuracyと、回答した症例だけのselective accuracyを分け、coverageを必ず併記します。これにより「不明を増やして見かけの精度を上げる」パイプラインを正しく比較できます。

 Git変更状況を確認した

設定と評価の実装が進みました。設定ファイルにAPI keyの値を直接書く形式は拒否し、環境変数名だけを許可します。またP5のensemble数不足、未知pipeline、未知設定キー、2を超える反論ラウンドも起動前に検出します。

 Git変更状況を確認した

runnerとCLIまで実装され、現在は標準ライブラリだけで全テストを再実行できるrunnerとREADMEを整備しています。添付CSVは変更せず、`validate-data` で2300行・100記録・23基準だけを出力する統合確認も行います。

 Git変更状況を確認した

依存なしテストrunnerでは、これまで直接実行できなかった `tmp_path` ・`monkeypatch` ・`capsys` 相当も標準ライブラリで再現し、全テストを一括実行します。pytestを入れた環境では通常のpytestも使える二重構成です。

 Review task5 完了

 確認しレビューした臨床試験QAタスクとパイプライン計画

CLI・評価・READMEの初回実装が完了し、99テストと添付CSVのread-only検証（2300行・100記録・23基準）を通過しました。現在、出力・manifest・CLI例外・config hashにPHIやAPI keyが混入しないかを独立レビューしています。

評価レビューでは、`note_id+criterion` の重複join、numeric blankと `not_specified` の矛盾、棄却を含むaccuracy、selective accuracy、MAE・tolerance・missingnessの分母を個別に確認しています。添付CSVの2件のblank/flag不一致は0にせず欠測として報告します。

レビューはrunnerの原子的書き込みまで追っています。途中失敗時に既存の完全なprediction fileを壊さないこと、tmp fileへflush・fsync後に置換すること、失敗メッセージに元カルテやAPI応答本文を含めないことを確認中です。

Task 5レビューで、成功時の評価JSONとCLI引数エラーにも自由入力が出し得る経路が見つかりました。評価CSVのcriterion/type/flagを固定23問仕様で検証し、argparseの既定エラー表示を安全なclass-only JSONへ置換します。併せてNaN設定、未使用client、prediction/manifestのrun単位原子性を修正します。

 Task5 cli eval 更新あり

設定topologyは未使用clientを許さない厳格なmatrixへ変更します。P1は単一primary、P2は複数primary+aggregator、P3はrole、P4は単一primary+verifier+任意router、P5は複数primary+aggregator+verifier+任意router、P6はrole+verifier+任意routerです。

run出力は、predictionとmanifestを別々に置換する方式から、両方を一時runディレクトリへ書いてfsync後、ディレクトリごと一度だけrenameする方式へ変えています。途中失敗では最終runが一切見えないため、メタデータ不一致を防げます。

 Git変更状況を確認した

評価側では、goldのcriterionを23仕様へ照合し、question_type不一致や未知 `not_specified` 表現をエラーにします。これで、自由記述が `by_criterion` のJSON keyとして成功出力へ流れる経路と、未知flagを0扱いする静かな誤変換を防ぎます。

 Rereview task5 fix1 完了

📄 臨床試験QAパイプライン作業をレビューした

Task 5修正後は110テストが通過しています。再レビューでは、成功時・失敗時とも自由入力を出さないこと、exact topology、NaN/Infinity拒否、runディレクトリの一括publish、status/answer正規化を個別に再確認しています。

🔥 Task5 cli eval 完了

残った問題は、CLIの `--pipeline` 上書き後にtopology再検証が走らない経路です。
`build_runtime()` 自身を最終防衛線にし、上書き後のclient構成が一致しなければ、client生成・API呼び出し・run作成の前に安全な設定エラーで停止させます。

🌱 Rereview task5 fix2 完了

🔗 臨床試験QAパイプライン変更をレビューしパッチ適用

全機能の実装レビューが完了しました。ここから、設計書・実装計画に対する全体コードレビューを最も厳しい条件で行い、その指摘を反映後に113テスト、compile、CLI、添付CSVのハッシュ不変をfreshに再検証します。

🌱 Final code review 完了

📄 臨床試験QA作業のレビューを準備した

全体レビューは15コミット・約25万byteの差分を対象に、PHI漏洩、根拠台帳、数値完全性、6 topology、設定・評価・atomic publish、README整合性を横断確認しています。配布前の最終ゲートなので、局所テストでは見えにくいモジュール間の不整合を重点的に見えています。

最終レビューでは、既知の「空行だけで構成された見出しなしノート」のルーティング粒度も再評価しています。これは全文fallbackで再現率は保たれますが、P4～P6のコスト削減効果に影響し得るため、release blockerか将来拡張かを切り分けます。

レビューは継続中です。特に、run出力が安全のため根拠quoteを保存しない一方、研究目的では根拠監査が必要というトレードオフについて、READMEの説明と評価可能範囲が一致しているかを確認しています。原文根拠そのものはメモリ内結果に保持し、永続化は件数・妥当性のみです。

処理中

ChatGPT の回答は必ずしも正しいとは限りません。重要な情報は確認するようにしてください。